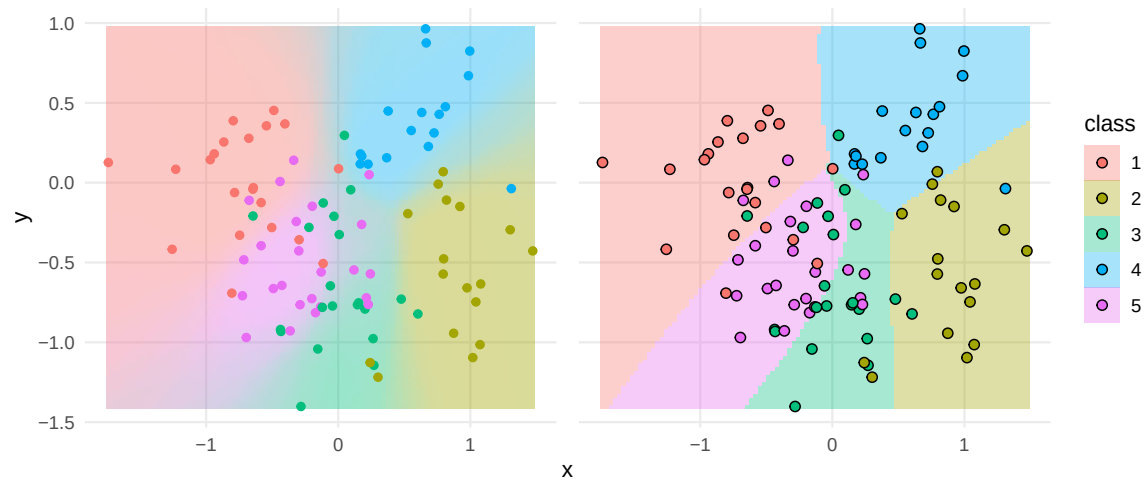


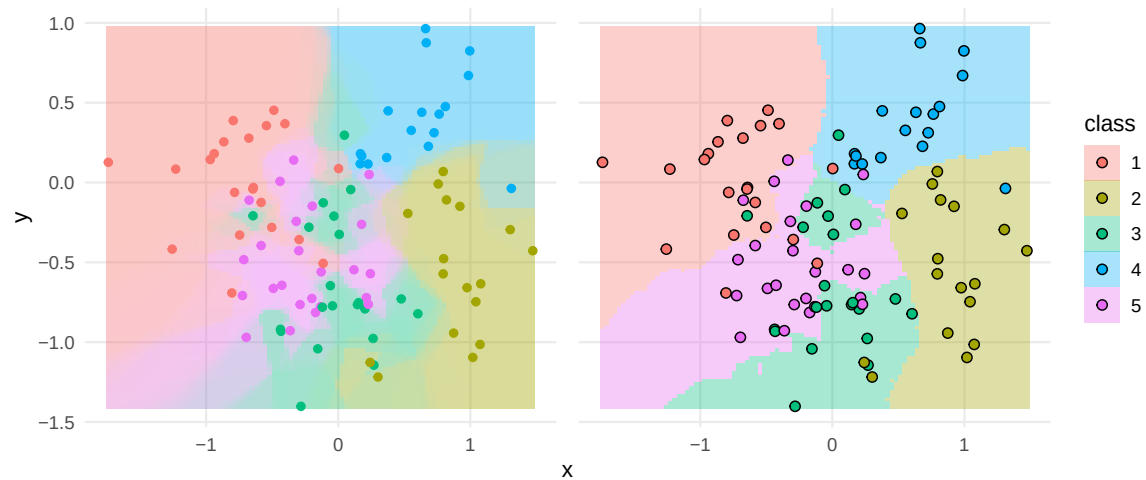
25-11-12_update

Placing classes with non continuous color pallets next to each other

```
classify_boot_blend(bidata, simpleplot, n = 100, res = 101, method = "qda")+  
  theme(legend.position = "none")+  
ggplot(bidata, aes(x = x, y = y, fill = class))+  
  geom_classify(method = "qda")+  
  geom_point_fill() + plot_layout(guides = "collect", axes = "collect")
```

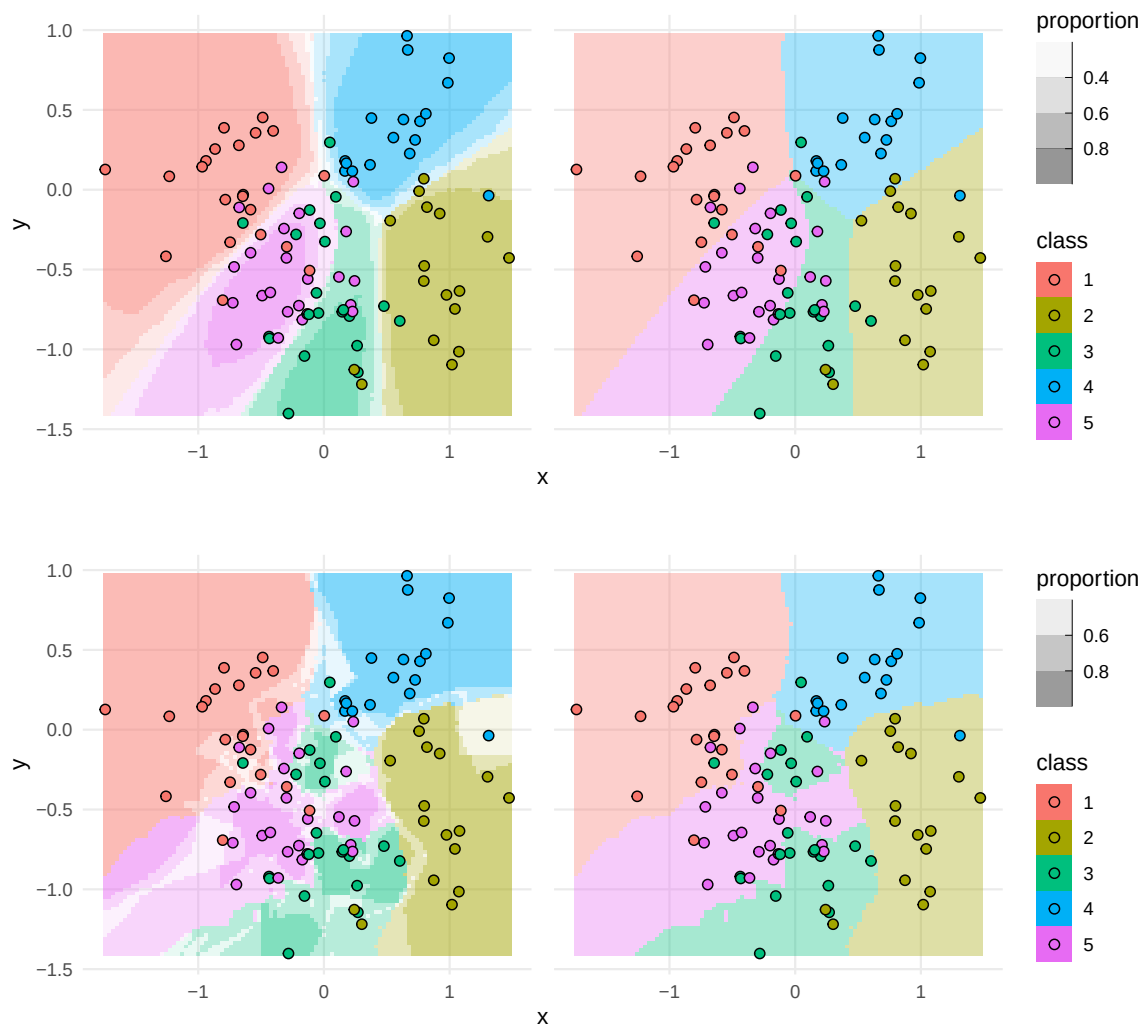


Same plot with KNN



Repeating those plots using the binned proportion of the plurality class.

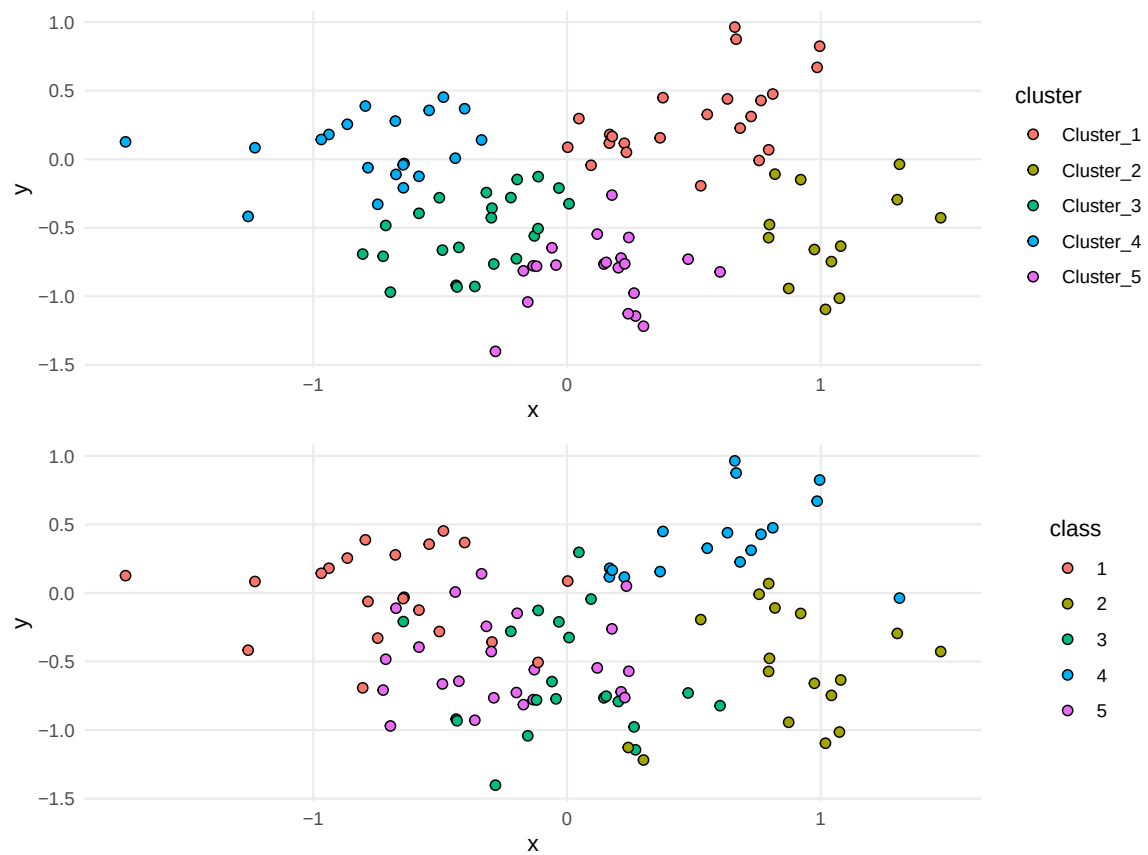
```
classify_boot_blend(bidata, simpleplot, n = 100, res = 101, method = "qda", style = "fade")+  
  
ggplot(bidata, aes(x = x, y = y, fill = class))+  
  geom_classify(method = "qda")+  
  theme(legend.position = "none")+  
  geom_point_fill() + plot_layout(guides = "collect", axes = "collect")
```



geom_point_cluster

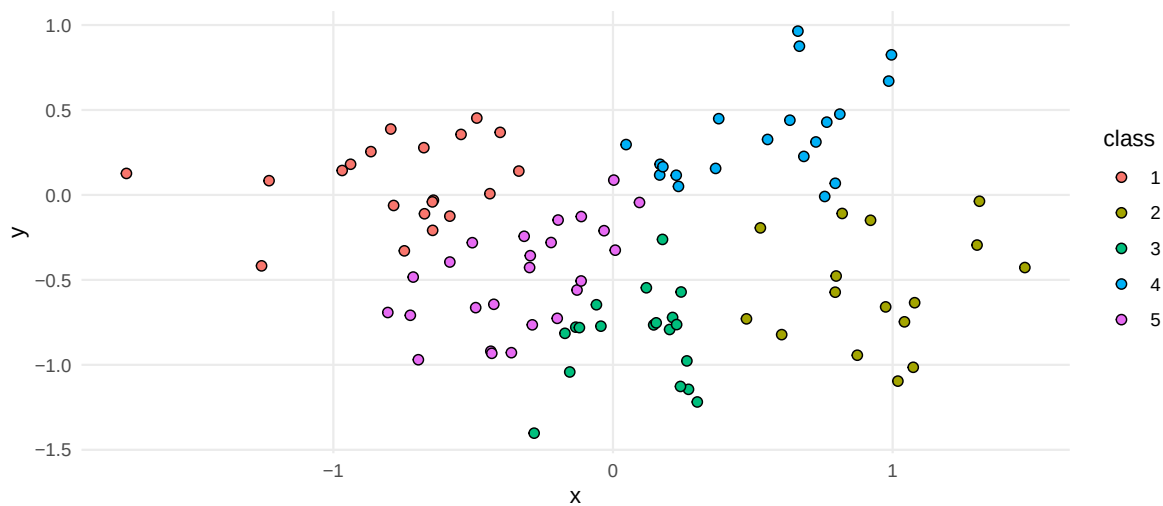
```
kmeans_spec_5 <- k_means(num_clusters = 5) |> set_engine("stats")

(ggplot(bidata, aes(x, y))+
  geom_point_cluster(method = kmeans_spec_5))/
(ggplot(bidata, aes(x, y))+
  geom_point_fill(aes(fill = class)))
```



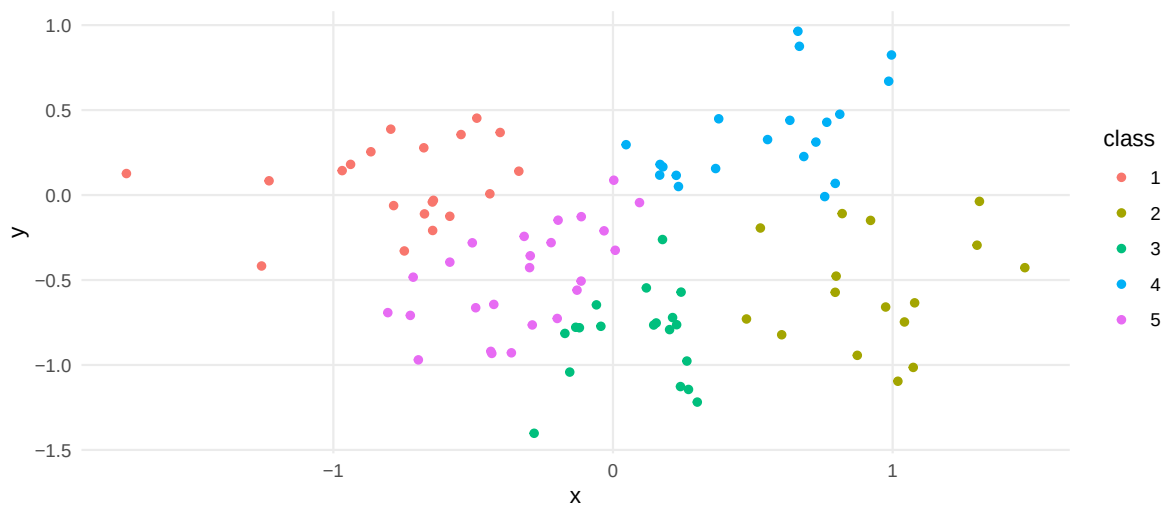
geom_point_classify

```
ggplot(bidata, aes(x, y, fill = class))+  
  geom_point_classify()
```



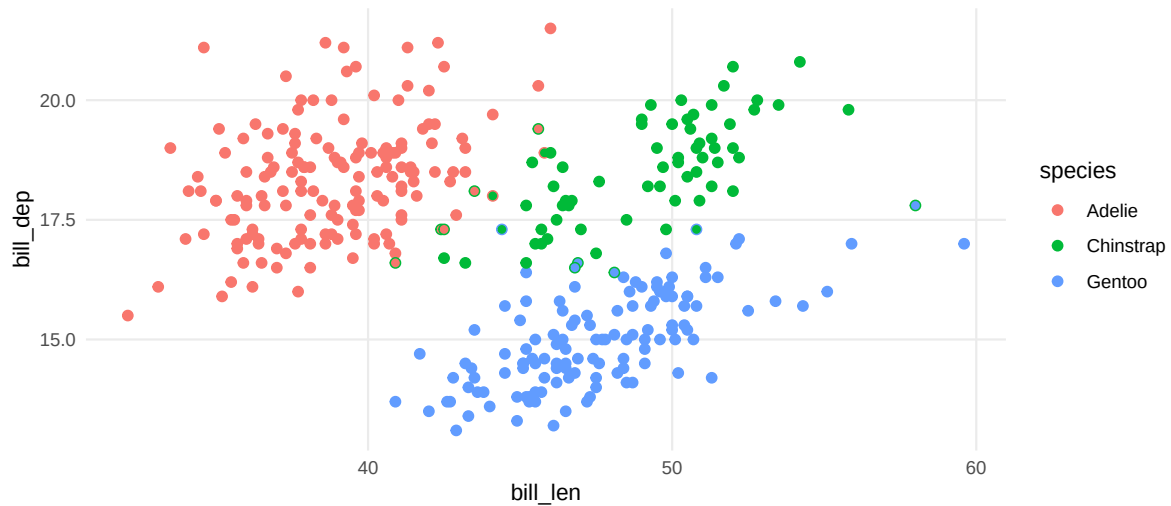
color can be used instead of fill via the class afterstat

```
ggplot(bidata, aes(x, y, fill = class)) +  
  stat_point_classify(geom = GeomPoint, aes(col = after_stat(class)), shape = 19)
```

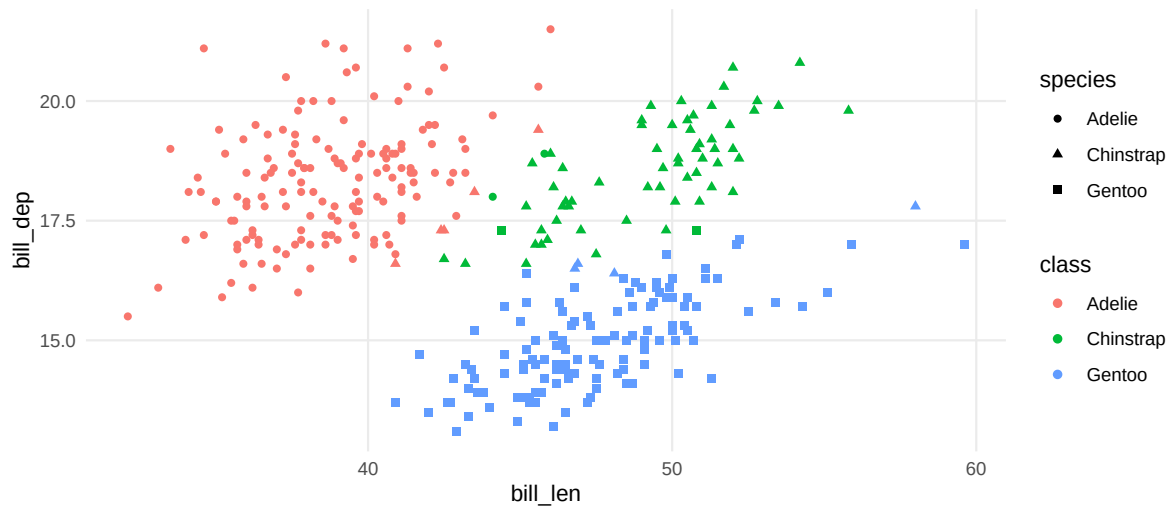


An interesting way to see where the classifier fails is combining color and fill, or color and shape

```
ggplot(penguins, aes(bill_len, bill_dep, fill = species, col = species)) +
  geom_point_classify()
```



```
ggplot(penguins, aes(bill_len, bill_dep, fill = species)) +
  stat_point_classify(geom = GeomPoint,
    aes(col = after_stat(class), shape = species))
```



Returning to a class aesthetic?

I went back through my previous implementation and was reminded of part of the reason we moved on from class as an aesthetic. It didn't properly produce a legend. The implementation looked like this.

```
Statclassify <- ggproto("Statclassify", Stat,
  setup_params = cpt_par,
  setup_data = cpt_data,
  compute_panel = cpt_class,
  required_aes = c("x", "y", "class"),
  default_aes = aes(fill = ..fill..)
)
```

I took in a class, but then internally computed a fill after stat and returned it. This plotted fine, but the legend didn't know to link class to fill. I tried to experiment with allowing for class and fill to both be supplied for `geom_point_classify`, but while i got it to plot fine, the legend wouldn't appear for class aesthetic.

Is kmean working?

Having read further on kmeans one of the two engines used is from the stats package of base R, so results should be the same. However there are a number of settings available to change that could improve performance.

comparing base and tidyclust

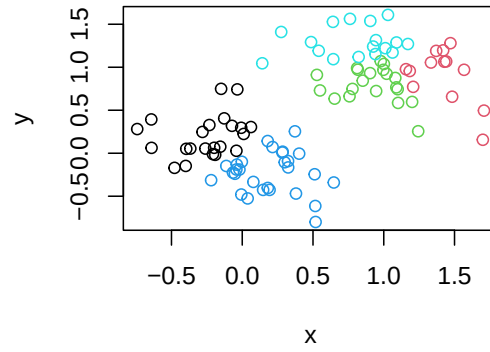
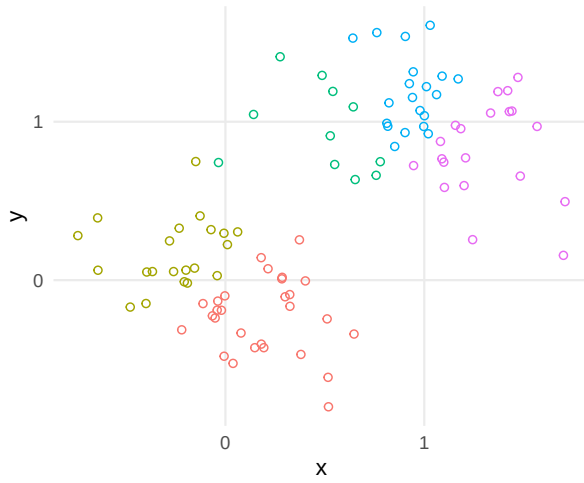
```
kmeans_spec_5 <- k_means(num_clusters = 5) |> set_engine("stats")

x <- rbind(matrix(rnorm(100, sd = 0.3), ncol = 2),
  matrix(rnorm(100, mean = 1, sd = 0.3), ncol = 2))
colnames(x) <- c("x", "y")

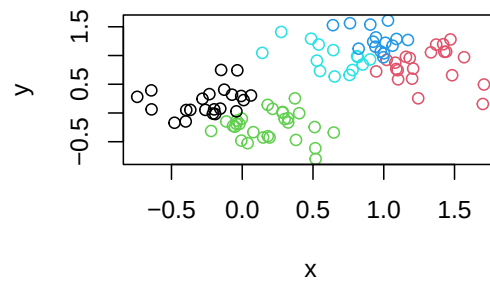
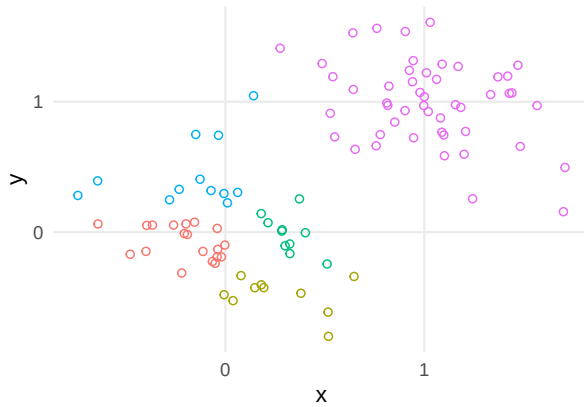
# Everything below this I reran before each plot.
cl <- kmeans(x, 5)
mod <- fit(kmeans_spec_5, ~x+y, data=data.frame(x))
df <- tibble(y = x[,2], x = x[,1], cluster = factor(extract_cluster_assignment(mod)$cluster))

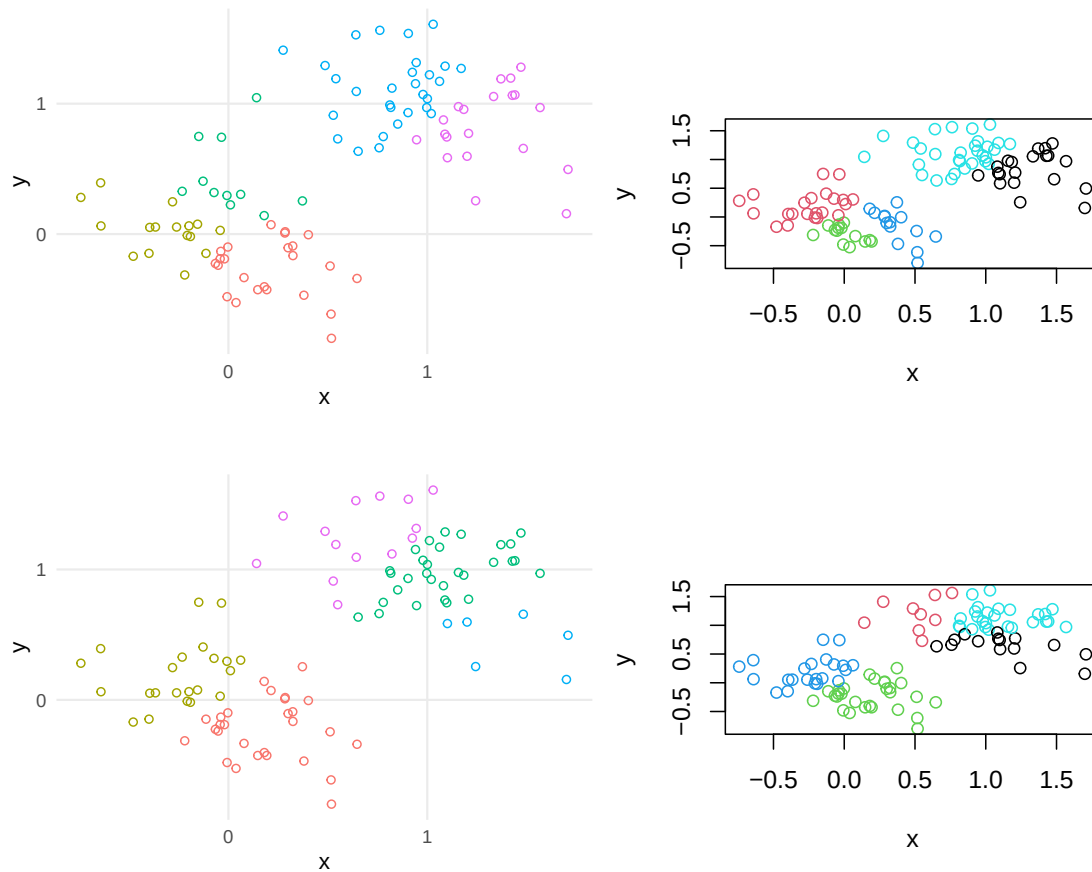
ggplot(df, aes(x,y))+
  geom_point(shape = 21, aes(col = cluster))+
```

```
theme(legend.position = "none")+
~plot(x, col = cl$cluster)
```



Further repetition





Across all these plots there are a couple variations in results for both implementations for tidyclust. They do appear to work the same

improving performance

It turns out the default parameters for kmeans are simply terrible, just by changing the number of starts to 10 instead of the default 1 performance seems much better

```
kmeans_spec_5_improved <- k_means(num_clusters = 5) |> set_engine("stats", nstart = 10)
```

